# AI-Assisted Financial Reconciliation: Governance & Testing Report

This comprehensive governance document certifies the testing, control framework, and validation logic developed for the **AI-Assisted Financial Reconciliation Engine** (Rule Version `1.0.0`, Batch `REC-5E09768EEF`).

---

## 1. Executive Summary

Manual balance sheet and ledger reconciliations represent a significant source of operational risk, bottlenecking month-end close cycles. By introducing automated matching rules alongside robust structural validation, this project achieves a **100% pass rate** across all defined User Acceptance Testing (UAT) scenarios while strictly enforcing human-in-the-loop oversight for complex or anomalous transactions.

### Core Performance Indicators

- **Total Transaction Records Processed**: 8 unique reference keys
- **Automated Match Rate**: 37.5% (3/8 unique references fully reconciled by rules)
- **Manual Exception Review Backlog**: 5 rows flagged (62.5% of total volume)
- **UAT Success Rate**: 100% (9 out of 9 synthetic validation scenarios passed)

---

## 2. Process Control & Governance Architecture

To ensure financial data integrity and comply with modern auditing standards, the system implements a multi-tiered control schema.

### Primary Control Standard Operating Procedures (SOPs):

1. **Input Stage (Structural Safeguards)**: Prior to running the matching engine, files are evaluated for structural completeness (presence of `transaction_id`, `date`, `amount`). Any missing fields, duplicate columns, empty files, or ill-formatted dates result in a **hard schema rejection** (`DataValidationError`).
2. **Processing Stage (Floating-Point Defense)**: Values in floating-point representations are strictly prohibited to prevent decimal rounding errors during evaluations. Currencies are scaled up to and stored as **integer cents** (using Python's `Decimal` module).
3. **Post-Processing Stage (Human-in-the-Loop)**: Any transactional status other than a direct, unique `Matched` status is assigned a governance reason code (e.g., `DUPLICATE_REFERENCE`, `AMOUNT_MISMATCH`) and routed directly to a pending review queue. **No automated entries are posted for these flagged records.**

---

## 3. Control Risk Register

- **CTRL-001 (Decimal Precision Protection)**: Values are converted into Python `Decimal` and represented internally as integer cents before comparison. (Risk Severity: **High** | Residual Risk: **Low**)
- **CTRL-002 (Zero-Trust Input Verification)**: `validate_transactions` performs strict null, datatype, and empty-set validation on ingestion. (Risk Severity: **Medium** | Residual Risk: **Low**)
- **CTRL-003 (Duplicate Reference Prevention)**: Grouping checks count occurrences on both sources. If a duplicate is detected, it is marked for `Duplicate review`. (Risk Severity: **High** | Residual Risk: **Low**)

- **CTRL-004 (Authorized Review Oversight)**: Automated ledger posting and payment functions remain explicitly disabled. (Risk Severity: **Critical** | Residual Risk: **Low**)

---

## 4. User Acceptance Testing (UAT) Dashboard

All 9 synthetic scenarios designed to stress-test validation rules, data conversions, boundary conditions, and matching behavior passed successfully.

- **UAT-01 (Same reference and amount)**: Matched -> **PASS**
- **UAT-02 (Same reference but different amount)**: Amount mismatch -> **PASS**
- **UAT-03 (Transaction exists only in bank statement)**: Missing in ledger -> **PASS**
- **UAT-04 (Transaction exists only in ledger)**: Missing in bank statement -> **PASS**
- **UAT-05 (Duplicate transaction reference)**: Duplicate review -> **PASS**
- **UAT-06 (Required column missing)**: Input rejected with validation error -> **PASS**
- **UAT-07 (Blank transaction reference)**: Input rejected with validation error -> **PASS**
- **UAT-08 (Invalid transaction date)**: Input rejected with validation error -> **PASS**
- **UAT-09 (Invalid transaction amount)**: Input rejected with validation error -> **PASS**

*UAT evidence records have been compiled and exported to `/content/governance_outputs/uat_evidence_tracker.csv`.*

---

## 5. Limitations & Responsible AI Boundary Constraints

1. **Synthetic Data Boundaries**: All tests and executions are performed strictly using sandboxed synthetic files. Passing local UAT does not

imply production readiness on scale, real-world bank feeds, or highly variable ERP schemas.

2. **Authority Limits**: The engine maps exception reasons but does **not** assume, simulate, or fabricate final reviewer decisions. All review logs generate as a template with a clean `PENDING_REVIEW` state.
3. **Schema Rigidity**: The system currently enforces exact matching on `transaction_id`. Fuzzy description matching or sliding-window date matching are intentionally excluded in Version 1.0.0 to prevent false-positives.

---

## 6. Business Impact & Efficiency Assumptions

- **Close Cycle Acceleration**: By auto-reconciling 37.5% of unique entries instantaneously, the finance team can redirect focus entirely to complex exception resolution, reducing overall close time.
- **Audit Readiness**: The automated generation of the immutable `reconciliation_results_governed.csv` and accompanying `exception_review_log.csv` provides internal and external auditors with clear traceability.
- **Reduced Write-offs**: Capturing amount discrepancies immediately (such as the 50.00 difference in `T003`) prevents compounding bookkeeping variances and premature write-offs.